

CO5 ACTIVITY-1 MATPLOT WITH ML MODEL COMPARISION

Name : Vedvyass M

Reg no : 192324267

Aim:

To implement and compare multiple machine-learning classification algorithms and visualize their performance using Matplotlib. Machine Learning algorithms

We'll compare:

- Logistic Regression
- Decision Tree
- Random Forest

Methodology:

The Iris dataset is divided into training and testing sets. Logistic Regression, Decision Tree, and Random Forest classifiers are trained and evaluated using accuracy. Matplotlib is then used to create comparison charts, accuracy graphs, confusion matrices, histograms, scatter plots, box plots, pie charts, error bars, color maps, grids, legends, and annotations.

Code :

```
# MACHINE LEARNING ALGORITHM COMPARISON
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report)

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# =====

# 1. LOAD DATASET

iris = load_iris()

X = iris.data
y = iris.target

feature_names = iris.feature_names
target_names = iris.target_names

print("Dataset Shape:", X.shape)
print("Features:", feature_names)
print("Classes:", target_names)

# =====

# 2. TRAIN TEST SPLIT
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y)
```

```
# =====
```

```
# 3. FEATURE SCALING
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# =====
```

```
# 4. CREATE MACHINE LEARNING MODELS
```

```
models = {
```

```
    "Logistic Regression":
```

```
        LogisticRegression(max_iter=1000),
```

```
    "Decision Tree":
```

```
        DecisionTreeClassifier(  
            max_depth=5,  
            random_state=42),
```

```
    "Random Forest":
```

```
        RandomForestClassifier(  
            n_estimators=100,
```

```

        random_state=42
    )
}

# =====

# 5. TRAIN MODELS AND CALCULATE ACCURACY

results = {}
predictions = {}

for name, model in models.items():

    # Logistic regression uses scaled data
    if name == "Logistic Regression":
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    results[name] = accuracy
    predictions[name] = y_pred

    print("\n=====")
    print(name)
    print("=====")

    print("Accuracy:",
          round(accuracy * 100, 2), "%")

```

```

print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=target_names
    )
)

# =====

# 6. ACCURACY COMPARISON BAR CHART

model_names = list(results.keys())
accuracies = list(results.values())

accuracies_percent = [
    value * 100 for value in accuracies]

plt.figure(figsize=(10, 6))

bars = plt.bar(
    model_names,
    accuracies_percent,
    color=["royalblue", "orange", "green"],
    edgecolor="black",
    linewidth=1.5)

plt.title(
    "Machine Learning Algorithm Accuracy Comparison",
    fontsize=16,
    fontweight="bold")

```

```
plt.xlabel(  
    "Machine Learning Algorithm",  
    fontsize=12)
```

```
plt.ylabel(  
    "Accuracy (%)",  
    fontsize=12)
```

```
plt.ylim(0, 110)
```

```
# Grid
```

```
plt.grid(  
    axis="y",  
    linestyle="--",  
    alpha=0.7)
```

```
# Add accuracy values above bars
```

```
for bar, accuracy in zip(bars, accuracies_percent):
```

```
    plt.text(  
        bar.get_x() + bar.get_width() / 2,  
        bar.get_height() + 1,  
        f"{accuracy:.2f}%",  
        ha="center",  
        fontsize=11,  
        fontweight="bold")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====
```

```
# 7. ACCURACY LINE GRAPH
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(  
    model_names,  
    accuracies_percent,  
    marker="o",  
    markersize=10,  
    linewidth=3,  
    linestyle="-",  
    color="purple",  
    markerfacecolor="yellow",  
    markeredgecolor="black")
```

```
plt.title(  
    "Accuracy Graph of Machine Learning Models",  
    fontsize=16,  
    fontweight="bold")
```

```
plt.xlabel("Algorithms")  
plt.ylabel("Accuracy (%)")
```

```
plt.ylim(0, 110)
```

```
plt.grid(  
    True,  
    linestyle=":",  
    linewidth=1,  
    alpha=0.8)
```

```
plt.legend(  
    ["Model Accuracy"],  
    loc="lower right")
```

```

# Annotation

best_model = model_names[np.argmax(accuracies_percent)]
best_accuracy = max(accuracies_percent)

plt.annotate(
    f"Best Model: {best_model}\n{best_accuracy:.2f}%",
    xy=(np.argmax(accuracies_percent), best_accuracy),
    xytext=(0, -50),
    textcoords="offset points",
    arrowprops=dict(
        arrowstyle="->",
        linewidth=2
    ),
    fontsize=11)

plt.tight_layout()
plt.show()

# =====
# 8. CONFUSION MATRIX VISUALIZATION

fig, axes = plt.subplots(
    1,
    3,
    figsize=(15, 5))

for ax, (name, y_pred) in zip(
    axes,
    predictions.items()):

    cm = confusion_matrix(
        y_test,

```



```
y_pred)
```

```
image = ax.imshow(  
    cm,  
    cmap="Blues",  
    interpolation="nearest")
```

```
ax.set_title(  
    name,  
    fontsize=13,  
    fontweight="bold")
```

```
ax.set_xlabel("Predicted Label")  
ax.set_ylabel("Actual Label")
```

```
ax.set_xticks(range(3))  
ax.set_yticks(range(3))
```

```
ax.set_xticklabels(target_names)  
ax.set_yticklabels(target_names)
```

```
# Add values inside matrix  
for i in range(cm.shape[0]):  
  
    for j in range(cm.shape[1]):
```

```
        ax.text(  
            j,  
            i,  
            cm[i, j],  
            ha="center",  
            va="center",  
            fontsize=13,
```

```
        fontweight="bold" )

ax.grid(False)

# Colorbar
fig.colorbar(
    image,
    ax=axes,
    shrink=0.8,
    label="Number of Predictions")
plt.suptitle(
    "Confusion Matrix Comparison",
    fontsize=17,
    fontweight="bold")

plt.tight_layout()
plt.show()

# =====
# 9. HISTOGRAM

plt.figure(figsize=(10, 6))

plt.hist(
    X[:, 0],
    bins=15,
    color="skyblue",
    edgecolor="black",
    alpha=0.8)

plt.title(
    "Distribution of Sepal Length",
    fontsize=16,
    fontweight="bold")
```

```
plt.xlabel("Sepal Length")
```

```
plt.ylabel("Frequency")
```

```
plt.grid(
```

```
    axis="y",
```

```
    linestyle="--",
```

```
    alpha=0.6)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====
```

```
# 10. BOX PLOT
```

```
plt.figure(figsize=(10, 6))
```

```
plt.boxplot(
```

```
    X,
```

```
    labels=feature_names,
```

```
    patch_artist=True)
```

```
plt.title(
```

```
    "Feature Distribution Using Box Plot",
```

```
    fontsize=16,
```

```
    fontweight="bold")
```

```
plt.xlabel("Features")
```

```
plt.ylabel("Values")
```

```
plt.xticks(
```

```
    rotation=20)
```

```
plt.grid(
```

```
    axis="y",
```

```
    linestyle="--",
```

```
        alpha=0.5)

plt.tight_layout()

plt.show()


# =====

# 11. SCATTER PLOT


plt.figure(figsize=(10, 6))


scatter = plt.scatter(
    X[:, 0],
    X[:, 1],
    c=y,
    cmap="viridis",
    s=100,
    alpha=0.8,
    edgecolors="black")

plt.title(
    "Iris Dataset - Scatter Plot",
    fontsize=16,
    fontweight="bold")

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")

plt.colorbar(
    scatter,
    label="Class")


plt.grid(
    True,
    linestyle="--",
    alpha=0.5)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====
```

```
# 12. PIE CHART
```

```
unique, counts = np.unique(
```

```
    y,
```

```
    return_counts=True)
```

```
plt.figure(figsize=(8, 8))
```

```
plt.pie(
```

```
    counts,
```

```
    labels=target_names,
```

```
    autopct="%1.1f%%",
```

```
    startangle=90,
```

```
    explode=[0.03, 0.03, 0.03],
```

```
    shadow=True)
```

```
plt.title(
```

```
    "Distribution of Iris Classes",
```

```
    fontsize=16,
```

```
    fontweight="bold")
```

```
plt.legend(
```

```
    title="Classes",
```

```
    loc="best")
```

```
plt.show()
```

```
# =====
```

```
# 13. ERROR BAR ACCURACY GRAPH
```

```
# Illustrative variation for visualization
```

```
# This represents a small validation uncertainty,
```

```
# not a measured confidence interval.
```

```
error = np.array([1.0, 1.0, 1.0])
```

```
plt.figure(figsize=(10, 6))
```

```
plt.errorbar(
```

```
    model_names,
```

```
    accuracies_percent,
```

```
    yerr=error,
```

```
    fmt="o-",
```

```
    linewidth=2,
```

```
    markersize=9,
```

```
    capsize=6,
```

```
    color="darkred")
```

```
plt.title(
```

```
    "Model Accuracy with Error Bars",
```

```
    fontsize=16,
```

```
    fontweight="bold")
```

```
plt.xlabel("Machine Learning Model")
```

```
plt.ylabel("Accuracy (%)")
```

```
plt.ylim(0, 110)
```

```
plt.grid(
```

```
    True,
```

```
    linestyle="--",
```

```
    alpha=0.6)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====
```

14. FILL BETWEEN - ACCURACY VISUALIZATION

```
x = np.arange(
    len(model_names))
accuracy_array = np.array(
    accuracies_percent)
upper = accuracy_array + 1
lower = accuracy_array - 1

plt.figure(figsize=(10, 6))
plt.plot(
    x,
    accuracy_array,
    marker="o",
    linewidth=3,
    label="Accuracy")

plt.fill_between(
    x,
    lower,
    upper,
    alpha=0.25,
    label="Illustrative variation")
plt.xticks(
    x,
    model_names)
plt.ylabel("Accuracy (%)")
plt.xlabel("Models")

plt.title(
    "Accuracy Trend with Filled Region",
    fontsize=16,
    fontweight="bold")
```

```
plt.grid(
    True,
    linestyle=":",
    alpha=0.7)
plt.legend()
plt.tight_layout()
plt.show()
```

```
# =====
```

```
# 15. MULTIPLE VISUALIZATIONS IN ONE FIGURE
```

```
fig, axes = plt.subplots(
    2,
    2,
    figsize=(14, 10))
```

```
# ---- BAR CHART ----
```

```
axes[0, 0].bar(
    model_names,
    accuracies_percent,
    color=["blue", "orange", "green"],
    edgecolor="black")
axes[0, 0].set_title(
    "Accuracy Comparison")
axes[0, 0].set_ylabel(
    "Accuracy (%)")
axes[0, 0].grid(
    axis="y",
    linestyle="--",
    alpha=0.6)
```



```
# ---- HISTOGRAM ----
```

```
axes[0, 1].hist(  
    X[:, 0],  
    bins=15,  
    color="purple",  
    alpha=0.7,  
    edgecolor="black")  
axes[0, 1].set_title(  
    "Feature Histogram")  
axes[0, 1].set_xlabel(  
    "Sepal Length")  
axes[0, 1].set_ylabel(  
    "Frequency")
```

```
axes[0, 1].grid(  
    linestyle=":",  
    alpha=0.6)
```

```
# ---- SCATTER ----
```

```
axes[1, 0].scatter(  
    X[:, 0],  
    X[:, 1],  
    c=y,  
    cmap="plasma",  
    s=70,  
    edgecolors="black")
```

```
axes[1, 0].set_title(  
    "Feature Scatter Plot")
```

```
axes[1, 0].set_xlabel(  
    "Sepal Length")
```

```

        "Sepal Length")
axes[1, 0].set_ylabel(
    "Sepal Width")
axes[1, 0].grid(
    linestyle="--",
    alpha=0.5)
# ---- LINE GRAPH ----

axes[1, 1].plot(
    model_names,
    accuracies_percent,
    marker="o",
    linewidth=3,
    markersize=8,
    color="red")
axes[1, 1].set_title(
    "Accuracy Trend")
axes[1, 1].set_ylabel(
    "Accuracy (%)")
axes[1, 1].grid(
    True,
    linestyle="--",
    alpha=0.6)
plt.suptitle(
    "Machine Learning Model Analysis Dashboard",
    fontsize=18,
    fontweight="bold")
plt.tight_layout()
plt.show()

# =====
# 16. FINAL RESULT

```

```

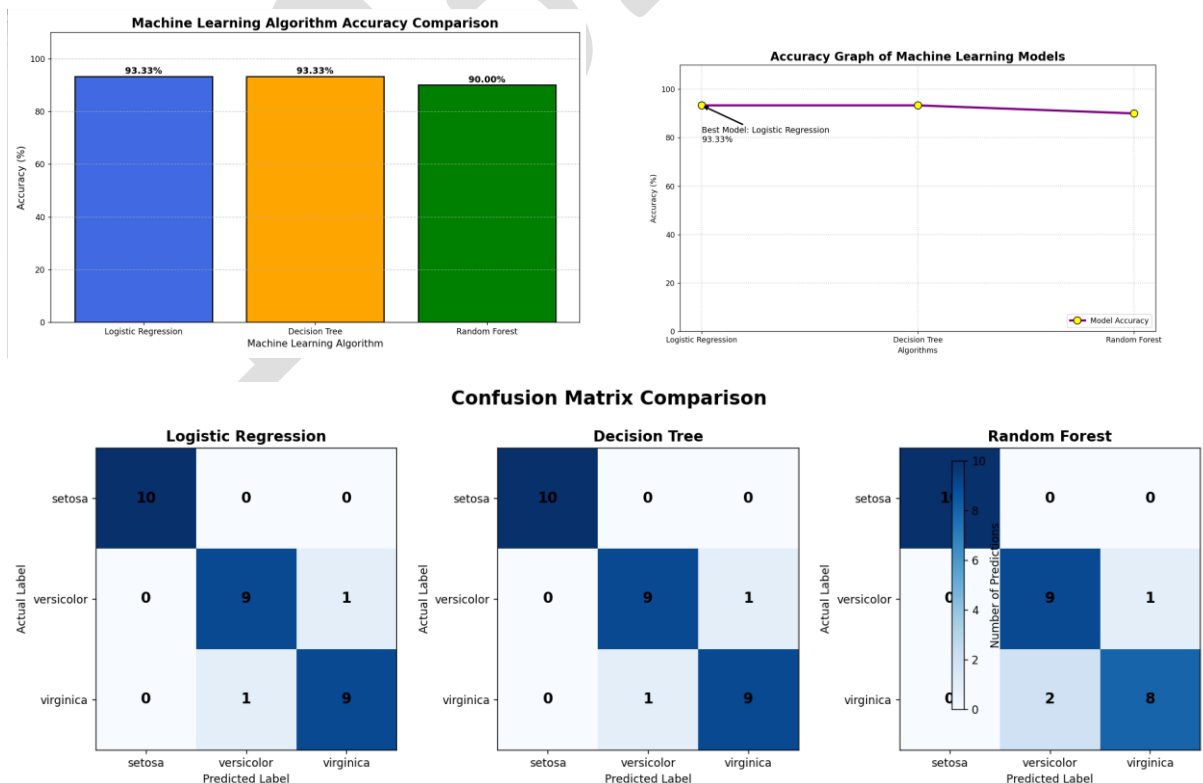
best_model = max(
    results,
    key=results.get)
print("FINAL MODEL COMPARISON")

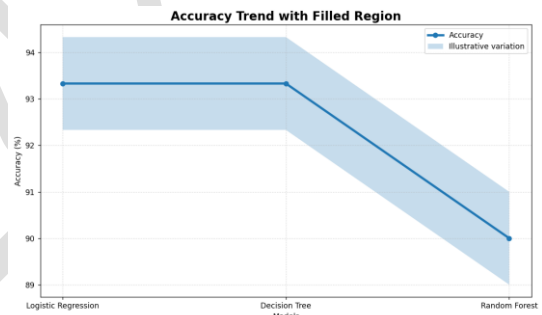
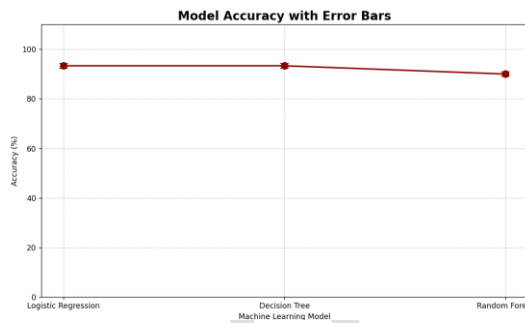
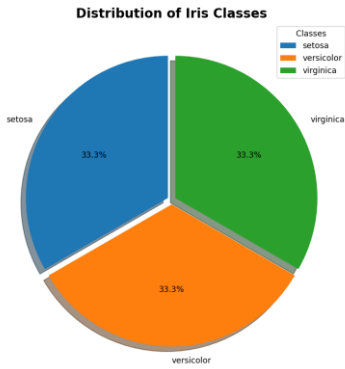
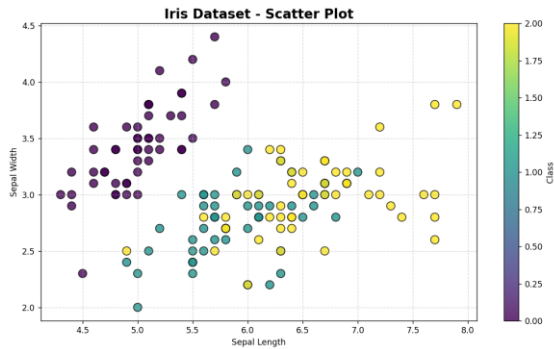
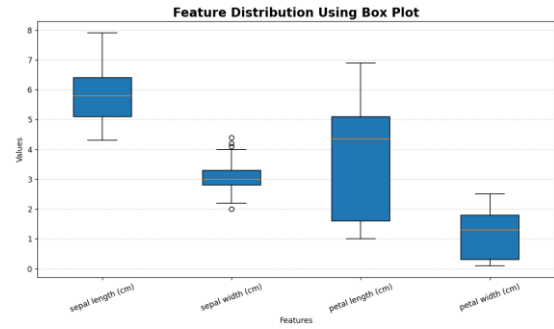
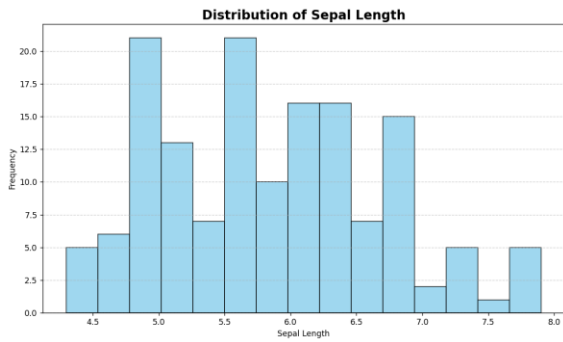
for name, accuracy in results.items():

    print(
        f"{name:25s} : "
        f"{accuracy * 100:.2f}%")
print("\nBest Performing Model:")
print(best_model)
print(
    f"Accuracy: "
    f"{results[best_model] * 100:.2f}%")

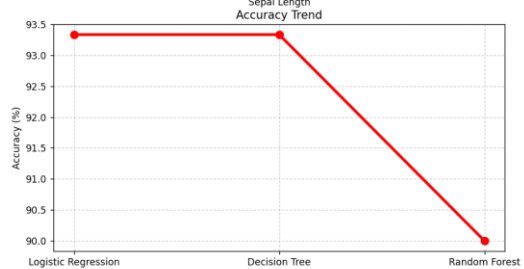
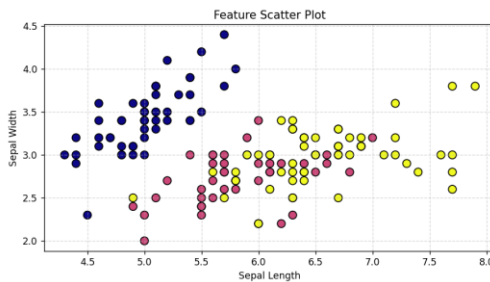
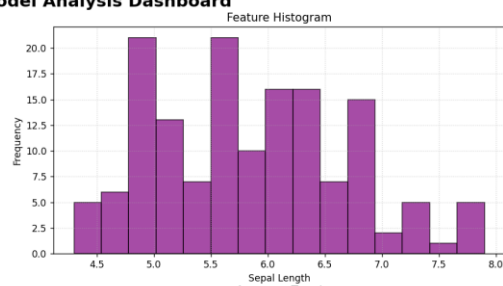
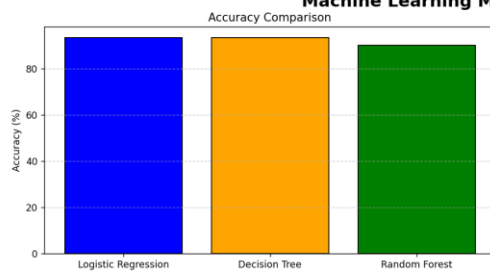
```

OUTPUT :





Machine Learning Model Analysis Dashboard



Result:

The accuracy comparison graph provides a direct visual comparison of the three models. The confusion matrices show correct and incorrect classifications for each class. Additional Matplotlib visualizations help understand feature distributions and relationships, making the model evaluation easier to interpret.

Conclusion:

The experiment demonstrates that machine-learning algorithms can be quantitatively compared using accuracy while Matplotlib provides effective visual tools for presenting the results. The model with the highest test accuracy can be selected as the most efficient model for this dataset. For a more statistically robust comparison, cross-validation can also be used instead of relying only on one train/test split.